

TASK 3 — Secure Coding Review

Project Title

Secure Coding Review of a Python Login System

1. Introduction

Secure coding is the practice of writing software in a way that protects systems and data from cyber threats and vulnerabilities. Poor coding practices can expose applications to attacks such as unauthorized access, password theft, data leaks, and code injection.

This project focuses on reviewing a simple Python login application to identify security weaknesses and recommend safer coding practices.

2. Objective

The objectives of this task are:

- To analyze insecure source code
- To identify security vulnerabilities in the application
- To understand risks associated with poor coding practices
- To implement secure coding improvements
- To recommend best practices for software security

3. Programming Language Used

- Python 3

4. Tools Used

- Visual Studio Code (VS Code)
- Python Interpreter
- Manual Code Inspection
- Bandit (Optional Static Analysis Tool)

5. Vulnerable Application Code

Insecure Login System

```
username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "1234":
    print("Login successful")
else:
    print("Access denied")
```

6. Security Vulnerabilities Identified

6.1 Hardcoded Credentials

Description

The username and password are directly stored in the source code.

Risk

If attackers access the source code, they can easily view the credentials and gain unauthorized access.

Recommendation

Store credentials securely using encrypted databases or environment variables.

6.2 Weak Password

Description

The password “1234” is extremely weak and easy to guess.

Risk

Attackers can crack weak passwords using brute-force attacks.

Recommendation

Use strong passwords containing:

- Uppercase letters
- Lowercase letters
- Numbers
- Special characters

Example:

`StrongPass@2026`

6.3 No Password Hashing

Description

Passwords are stored and compared in plain text.

Risk

Plain text passwords can be stolen easily if the system is compromised.

Recommendation

Hash passwords using secure algorithms such as:

- SHA-256
- bcrypt
- Argon2

6.4 Lack of Input Validation

Description

User inputs are accepted without validation.

Risk

This may expose the system to malicious input attacks.

Recommendation

Validate and sanitize all user inputs before processing.

6.5 No Account Protection Mechanism

Description

The system allows unlimited login attempts.

Risk

Attackers can perform brute-force attacks to guess passwords.

Recommendation

Implement:

- Login attempt limits
- CAPTCHA
- Temporary account lockouts

7. Improved Secure Version

```
import hashlib
```

```
stored_username = "admin"
```

```
stored_password = hashlib.sha256(
    "StrongPassword123".encode()
).hexdigest()

username = input("Enter username: ")
password = input("Enter password: ")

hashed_password = hashlib.sha256(
    password.encode()
).hexdigest()

if username == stored_username and hashed_password == stored_password:
    print("Login successful")
else:
    print("Access denied")
```

8. Security Improvements Implemented

The improved version includes:

- Password hashing using SHA-256
- Stronger password protection
- Better credential handling
- Improved authentication security

9. Best Practices for Secure Coding

Developers should follow these secure coding practices:

- Never hardcode passwords
- Use encryption and hashing
- Validate all user input
- Apply least privilege access
- Keep software updated
- Use multi-factor authentication
- Perform regular security testing
- Conduct code reviews frequently

10. Conclusion

This secure coding review demonstrated how simple coding mistakes can create major security vulnerabilities. By identifying insecure practices and implementing safer alternatives, developers can significantly reduce cybersecurity risks and improve software protection.

Secure coding is an essential practice for building reliable and resilient applications in today's digital environment.

11. Future Improvements

Future enhancements may include:

- Database integration
- Multi-factor authentication
- Password salting
- Logging and monitoring
- Secure session management
- Role-based access control

12. References

- OWASP Secure Coding Practices
- Python Documentation
- NIST Cybersecurity Framework
- Bandit Python Security Tool